

Documentación del código — Test 2 (S_n , función característica empírica)

Proyecto 2: Estadística No Paramétrica

Equipo: Gabriel Sánchez, Juan David Duarte, Luis Tejón, Claudio (IA)

Mayo 2026

*Este documento describe el código implementado para el **Test 2** del Proyecto 2: el test bootstrap de simetría basado en el estadístico de penalización S_n sobre la función característica empírica (Csörgő & Heathcote 1987; Feuerverger & Mureika 1977). Cubre el fundamento matemático, la simplificación numérica usada, la descripción módulo a módulo y las instrucciones para reproducir las simulaciones.*

Índice

1. Visión general	2
1.1. Estimadores del centro de simetría	2
1.2. Funciones de peso $w(t)$ implementadas	2
2. Simplificación numérica	2
3. Estructura del repositorio (parte S_n)	3
4. Módulos del paquete	3
4.1. <code>characteristic_fn.py</code>	3
4.2. <code>bootstrap_sn.py</code>	4
4.3. <code>simulation_sn.py</code> y <code>simulation_sn_parallel.py</code>	4
4.4. <code>plotting_sn.py</code>	5
5. Scripts ejecutables	5
5.1. Versión paralelizada (recomendada)	5
6. Salidas generadas	5
6.1. Datos	6
6.2. Figuras	6
7. Tests unitarios	6
8. Dependencias	7
9. Reproducir la simulación	7
10. Notas de diseño y comparación con T_n	7

1 Visión general

El Test 2 evalúa la misma hipótesis nula que el Test 1,

$$H_0 : F \text{ es simétrica respecto a algún } \theta,$$

pero la métrica de discrepancia se construye sobre la función característica empírica

$$c_n(t) = \frac{1}{n} \sum_{j=1}^n e^{itX_j}$$

en lugar de sobre la cdf empírica. La motivación es el Teorema: $X \sim \text{sim}(\theta)$ si y solo si $\varphi_X(t) = e^{it\theta} c_0(t)$ con $c_0 : \mathbb{R} \rightarrow \mathbb{R}$. En consecuencia, bajo H_0 la función $c_n(t) e^{-it\hat{\theta}}$ debe ser aproximadamente real.

Tomando como “simetrización” $c_s(t) = |c_n(t)|$ (misma magnitud, parte imaginaria nula), el estadístico de penalización es

$$S_n(\hat{\theta}) = \int_{-\infty}^{\infty} |c_n(t) e^{-it\hat{\theta}} - c_s(t)|^q w(t) dt, \quad (1)$$

con $q \in \{1, 2\}$ y w una densidad simétrica. Bajo H_0 la parte imaginaria de $c_n(t) e^{-it\hat{\theta}}$ se anula y S_n es pequeño; bajo asimetría, S_n crece.

1.1 Estimadores del centro de simetría

- **argmin:** $\hat{\theta} = \arg \min_{\theta} S_n(\theta)$ — minimiza la propia penalización.
- **mediana:** $\hat{\theta} = \text{median}(X_1, \dots, X_n)$.
- **media afeitada:** $\hat{\theta} = \overline{X}_{\alpha}$ con $\alpha = 0,1$.

1.2 Funciones de peso $w(t)$ implementadas

Nombre	Forma	Soporte efectivo
gauss_1.0	$w(t) = \phi_1(t) = \frac{1}{\sqrt{2\pi}} e^{-t^2/2}$	$ t \leq 6$
gauss_0.5	$w(t) = \phi_{1/2}(t)$ (más concentrada)	$ t \leq 3$
laplace_1.0	$w(t) = \frac{1}{2} e^{- t }$	$ t \leq 10$
cauchy_1.0	$w(t) = \frac{1}{\pi(1+t^2)}$	$ t \leq 40$
unif(T)	$w(t) = \frac{1}{2T} \mathbf{1}_{[-T, T]}(t)$	$ t \leq T$

2 Simplificación numérica

Escribimos $c_n(t) e^{-it\hat{\theta}} = A_n(t; \hat{\theta}) + iB_n(t; \hat{\theta})$ con

$$A_n(t; \theta) = \frac{1}{n} \sum_j \cos(t(X_j - \theta)),$$
$$B_n(t; \theta) = \frac{1}{n} \sum_j \sin(t(X_j - \theta)).$$

Como $c_s(t) = |c_n(t)| = \sqrt{A_n^2 + B_n^2}$ es real y positivo,

$$|c_n(t)e^{-it\hat{\theta}} - c_s(t)|^2 = (A_n - \sqrt{A_n^2 + B_n^2})^2 + B_n^2 = 2|c_n(t)|(|c_n(t)| - A_n(t; \hat{\theta})).$$

Esto evita raíces complejas y separa la dependencia de θ . Una segunda simplificación es:

$$A_n(t; \theta) = a_n(t) \cos(t\theta) + b_n(t) \sin(t\theta), \quad a_n(t) = \frac{1}{n} \sum_j \cos(tX_j), \quad b_n(t) = \frac{1}{n} \sum_j \sin(tX_j),$$

y $|c_n(t)|^2 = a_n^2 + b_n^2$ no depende de θ . La utilidad práctica es enorme: para el estimador **argmin** se barre θ sobre una grilla; precomputando (a_n, b_n) una sola vez, cada evaluación de $S_n(\theta)$ cuesta $O(K)$ en lugar de $O(nK)$.

Cuadratura

La integral se aproxima con regla trapezoidal sobre un grid uniforme $\{t_k\} \subset [-t_{\text{máx}}, t_{\text{máx}}]$. La cota $t_{\text{máx}}$ se fija automáticamente por cada w (Gaussiana: 6σ ; Laplace: $10/a$, etc.). Por defecto $K = 301$ puntos; en `-quick`, $K = 201$. Verificamos en tests que las densidades integran ≈ 1 .

3 Estructura del repositorio (parte S_n)

```
src/
|-- characteristic_fn.py          # S_n, pesos w(t), argmin_Sn
|-- bootstrap_sn.py              # Procedimiento bootstrap para S_n
|-- simulation_sn.py              # Estudio MC secuencial
|-- simulation_sn_parallel.py     # Estudio MC paralelo
|-- plotting_sn.py               # Graficas S_n y comparativas T_n vs S_n

scripts/
|-- run_test2_simulation_parallel.py # Script CLI principal

tests/
|-- test_characteristic_fn.py     # Tests unitarios
```

4 Módulos del paquete

4.1 characteristic_fn.py

Define los objetos centrales: la dataclass `WeightFn`, los factorías de pesos, el grid de integración, las dos versiones del estadístico (S_n y S_n multi- θ) y los estimadores del centro.

Núcleo: `Sn_multi_theta(sample, thetas, w_fn, q, t_grid)`. Implementa la simplificación descrita en §2: precomputa $(a_n, b_n, |c_n|)$ una sola vez y barre los M valores de θ en una sola operación vectorizada.

```
1 def Sn_multi_theta(sample, thetas, w_fn, q=2, t_grid=None):
2     if t_grid is None:
3         t_grid = make_t_grid(w_fn)
4     x = np.asarray(sample, dtype=float)
5     th = np.atleast_1d(np.asarray(thetas, dtype=float))
6
```

```

7  # c_n(t) raw
8  arg_raw = t_grid[:, None] * x[None, :]
9  a_n = np.mean(np.cos(arg_raw), axis=1)
10 b_n = np.mean(np.sin(arg_raw), axis=1)
11 abs_cn = np.sqrt(a_n**2 + b_n**2)
12
13 # A_n(t; theta) para todos los theta de una vez
14 tth = t_grid[:, None] * th[None, :]
15 A = a_n[:, None] * np.cos(tth) + b_n[:, None] * np.sin(tth)
16
17 # |c_n e^{-it.theta} - c_s|^2 = 2|c_n|(|c_n| - A)
18 abs_cn_col = abs_cn[:, None]
19 diff_sq = 2.0 * abs_cn_col * (abs_cn_col - A)
20 np.maximum(diff_sq, 0.0, out=diff_sq)
21
22 integrand = diff_sq if q == 2 else (
23     np.sqrt(diff_sq) if q == 1 else diff_sq**(q/2.0))
24
25 weights = w_fn.fn(t_grid[:, None])
26 return np.trapezoid(integrand * weights, t_grid, axis=0)

```

Listing 1: Vectorización de $S_n(\theta)$

Estimadores. Los tres se mantienen consistentes con el Test 1:

- `theta_median`: `np.median`.
- `theta_trimmed`: `scipy.stats.trim_mean`.
- `theta_argmin_Sn(sample, w_fn, q)`: búsqueda en 2 etapas (grid uniforme de 40 puntos + refinamiento Brent), aprovechando `Sn_multi_theta` para el barrido del grid.

4.2 bootstrap_sn.py

Implementa `bootstrap_test_Sn(sample, w_fn, q, estimator, B, t_grid, rng)` con el mismo flujo del Test 1, adaptado a S_n :

1. Estima $\hat{\theta}$ y calcula $S_{\text{obs}} = S_n(X; \hat{\theta}, q, w)$.
2. Construye el soporte simetrizado $\{X_i, 2\hat{\theta} - X_i\}$.
3. Para $b = 1, \dots, B$: remuestrea, recalcula $\hat{\theta}^*$ y obtiene $S_n^*(b)$.
4. $p = \frac{1 + \#\{S_n^*(b) \geq S_{\text{obs}}\}}{B+1}$.

El resultado es un `BootstrapResultSn` con: `statistic`, `p_value`, `theta_hat`, `boot_statistics`, `estimator_name`, `weight_name`, `q`, `B`.

4.3 simulation_sn.py y simulation_sn_parallel.py

`SimConfigSn` extiende la configuración con las dimensiones adicionales: `qs`, `weight_names`, `n_t_grid`.

```

1  @dataclass
2  class SimConfigSn:
3      sample_sizes : tuple = (20, 40, 80, 160)
4      estimators    : tuple = ("argmin", "median", "trimmed")
5      qs            : tuple = (1, 2)
6      weight_names  : tuple = ("gauss_1.0", "gauss_0.5", "laplace_1.0")
7      B             : int  = 500

```

```

8      R          : int = 200
9      alpha      : float = 0.05
10     seed       : int = 2026
11     n_t_grid   : int = 301

```

`run_simulation_sn_parallel` reutiliza el registro `_SPEC_REGISTRY` del Test 1 (`'simulation_parallel.py'`) para serializar las `DistSpec` entre procesos. Cada tarea es una réplica completa con seed único `= seed + task_idx`.

4.4 plotting_sn.py

Genera 8 tipos de figuras en `results/figures/`:

Función	Salida
<code>plot_type_i_error_sn</code>	<code>sn_type_i_error_q{q}_{w}.png</code> (una por (q, w))
<code>plot_power_curves_sn</code>	<code>sn_power_curves_q{q}_{w}.png</code> (una por (q, w))
<code>plot_q1_vs_q2</code>	<code>sn_compare_q.png</code>
<code>plot_weights_compare</code>	<code>sn_compare_weights.png</code>
<code>plot_power_heatmap</code>	<code>sn_power_heatmap.png</code>
<code>plot_runtime_sn</code>	<code>sn_runtime.png</code>
<code>plot_pvalue_distribution_h0_sn</code>	<code>sn_pvalue_h0.png</code>
<code>plot_tn_vs_sn_power</code>	<code>compare_tn_vs_sn_power.png</code>
<code>plot_tn_vs_sn_runtime</code>	<code>compare_tn_vs_sn_runtime.png</code>

Las comparativas `compare_tn_vs_sn_*.png` solo se generan si existe `tn_simulation_summary.csv` en disco (i.e., el Test 1 ya fue corrido).

5 Scripts ejecutables

5.1 Versión paralelizada (recomendada)

```

python scripts/run_test2_simulation_parallel.py      # full
python scripts/run_test2_simulation_parallel.py --quick  # rápido (~7-10 min)

```

Comparación full vs -quick.

Parámetro	full	-quick
B (remuestras)	500	199
R (réplicas MC)	200	60
Tamaños n	{20, 40, 80, 160}	{20, 40, 80}
q	{1, 2}	{1, 2}
Pesos w	gauss_1.0, gauss_0.5, laplace_1.0	gauss_1.0, laplace_1.0
K (grid integración)	301	201
Tareas totales	72 000	10 800
Tiempo aprox. (12 cores)	~60–90 min	~7–10 min

6 Salidas generadas

6.1 Datos

- `results/data/sn_simulation_raw.csv`: una fila por réplica MC.
- `results/data/sn_simulation_summary.csv`: una fila por celda (`dist`, n , `estimator`, q , `weight`).

Columnas del CSV `raw`.

Columna	Descripción
<code>dist</code> , <code>under_h0</code>	Distribución y bandera H_0
<code>n</code>	Tamaño muestral
<code>estimator</code>	<code>argmin</code> , <code>median</code> o <code>trimmed</code>
<code>q</code>	1 o 2
<code>weight</code>	Nombre del peso $w(t)$
<code>rep</code>	Índice de la réplica MC
<code>statistic</code>	S_n observado
<code>p_value</code>	p-valor bootstrap
<code>reject</code>	<code>p_value</code> < α
<code>theta_hat</code>	Centro estimado
<code>time_s</code>	Tiempo del test (B remuestras incluidas)

El CSV `summary` añade: `reject_rate`, `mean_pvalue`, `mean_stat`, `mean_time_s`, `sd_time_s`, `n_rep`, `se_rate`.

6.2 Figuras

Para cada (q, w) se generan 2 figuras (Error Tipo I y curvas de potencia, $|q| \times |w| = 6$ figuras de cada tipo), más 7 figuras agregadas (comparativas, heatmap, runtime, p-valor bajo H_0 , y dos comparativas con T_n).

7 Tests unitarios

Ubicación: `tests/test_characteristic_fn.py`. Verifica:

- `Sn vs reference`: la versión vectorizada coincide con la implementación ingenua (suma sobre j explícita) para varios q y θ .
- `Sn translation invariance`: $S_n(X + c; \theta + c) = S_n(X; \theta)$.
- `Sn_multi vs loop`: `Sn_multi_theta` coincide con iterar `Sn_statistic`.
- `argmin recovers center`: con $n = 400$ normales, `theta_argmin_Sn` recupera el centro a $\pm 0,3$.
- `median/trimmed`: estimadores convergen al centro real.
- `weights integrate to 1`: las densidades $w(t)$ integran ≈ 1 sobre el soporte usado.
- `bootstrap H0 level`: bajo H_0 la tasa de rechazo es razonable.
- `bootstrap power gamma`: bajo H_a (Gamma) hay potencia $> 0,25$.

Ejecutar:

```
python tests/test_characteristic_fn.py
```

8 Dependencias

Las mismas que el Test 1 (numpy, scipy, pandas, matplotlib). Probado con Python 3.11.

9 Reproducir la simulación

1. Tests unitarios:

```
python tests/test_characteristic_fn.py
```

2. Sanity check rápido (7–10 min):

```
python scripts/run_test2_simulation_parallel.py --quick
```

3. Ejecución completa (60–90 min con 12 cores):

```
python scripts/run_test2_simulation_parallel.py
```

4. Resultados: datos en results/data/, figuras en results/figures/. Si tn_simulation_summary.csv existe, el script también produce compare_tn_vs_sn_*.png.

Reproducibilidad

Semilla por defecto: `seed=2026`. Cada réplica recibe un seed único derivado (`seed + task_idx`). Los resultados son *exactos* entre corridas con la misma configuración.

10 Notas de diseño y comparación con T_n

Costo computacional. T_n es $O(n)$ por evaluación (búsqueda sobre el grid de $2n$ puntos vía `searchsorted`); S_n es $O(nK)$ con $K \approx 300$ por evaluación. Empíricamente: S_n con estimador `median` es $\sim 5\text{--}10\times$ más lento que T_n con `median`, y con `argmin` la relación cae a $\sim 2\text{--}3\times$ porque el barrido vectorizado de θ amortiza muy bien el costo.

Sensibilidad a w y q . El Test 2 introduce dos hiper-parámetros (w, q) que no existen en T_n . Esto le da flexibilidad para adaptarse a tipos específicos de alternativa, pero requiere exploración. En la práctica:

- $q = 2$ amplifica desviaciones grandes; $q = 1$ es más robusto (en presencia de outliers o colas pesadas como Cauchy).
- Pesos w concentrados cerca de 0 (`gauss_0.5`) favorecen alternativas con asimetría visible en las frecuencias bajas (translaciones de masa, asimetría suave).
- Pesos más anchos (`laplace_1.0`) capturan asimetrías de cola (Pareto, Weibull con k pequeño).

Naturaleza del estadístico. A diferencia de T_n , S_n no es exactamente cero en muestras simétricas perfectas (porque $c_n(t)$ puede ser negativo en algunos t , y $c_s(t) = |c_n(t)|$ captura ese signo). Esto es comportamiento esperado: el bootstrap absorbe esta dispersión bajo H_0 y el test sigue siendo válido.

*Documento generado a partir del código presente en el repositorio del proyecto.
Ver también `documentacion_test1.pdf` para la documentación del Test 1.*